计算机组成原理实验报告

CPU Verilog

计试 2301 李奕博 2231411746
计试 2301 刘添毅 2236114918

2026 年 1 月 1 日

1 实验简介

1.1 实验内容介绍

本实验为西安交通大学 COMP461805 课程，《计算机组成原理》的实验，主要目标是使用 *Verilog* 实现一个简单的 CPU 数据通路，包括**单周期、多周期和流水线**三种实现方式。详细的代码文件与测试结果已经部署到 github 仓库 CPU-Verilog。

实验目标如下：

- 理解并设计单周期、多周期和流水线 CPU 的数据通路。
- 自学 Verilog HDL 语言，了解其基本语法和模块设计，之后仿真上述三种 CPU 数据通路。
- 在上述基础上，添加新指令以丰富数据通路的功能。

1.2 实验环境

本实验使用的编程语言、模拟器、编译软件如下：

- 编程语言：*Verilog*、*SystemVerilog*
- 模拟器：*ModelSim*（在 *Windows11* 操作系统上运行）、*GTKwave*（在 *MacOS Sequoia* 系统上运行）
- 编译软件：*Mars*

2 单周期 CPU 实现

2.1 单周期 CPU 简介

单周期 CPU 是指每条指令在**一个时钟周期**内完成**取指、译码、执行、访存、写回**五个阶段。其核心思想是将所有操作组合在一个周期内完成，因此控制逻辑简单，但时钟周期受最长指令路径限制。

本文中实现的单周期 cpu 支持 *MIPS* 指令集中的大部分指令，并做了一定的**扩展**。数据通路中，包含寄存器堆、ALU、指令与数据存储器、控制器等核心模块。同时，也为扩展的指令增加了自定义模块，详细信息请参考2.4。

2.2 实现的指令集

本文实现的单周期 CPU 数据通路支持共 **89 条指令**，实现了 *MIPS* 指令集中的大部分指令，同时扩展了一部分指令，主要集中在条件转移部分，详细的指令如下表格所示：

功能分类	助记符与功能
加载	LW（加载字）
保存	SW（存储字）
R–R 运算	ADD（加），ADDU（无符号加），SUB（减），SUBU（无符号减），SLL（逻辑左移），SRL（逻辑右移），SRA（算术右移），AND（与），OR（或），XOR（异或），NOR（或非），SLT（有符号小于置 1），SLTU（无符号小于置 1）
R–I 运算	ADDI（加立即数），ADDIU（无符号加立即数），ANDI（与立即数），ORI（或立即数），XORI（异或立即数），LUI（加载立即数至高位），SLTI（小于立即数置 1），SLTIU（无符号小于立即数置 1）
分支	BEQ（等于 0 则分支），BNE（不等于 0 则分支），BLEZ（小于等于 0 则分支），BGTZ（大于 0 则分支），BLTZ（小于 0 则分支），BGEZ（大于等于 0 则分支）
跳转	J（跳转），JAL（跳转并链接），JALR（跳转并链接寄存器），JR（跳转至寄存器）
有条件跳转一	JE，JNE，JA，JNA，JB，JNB，JG，JNG，JL，JNL，JS，JNS，JO，JNO

功能分类	助记符与功能
有条件跳转二	JALE，JALNE，JALA，JALNA，JALB，JALNB，JALG，JALNG，JALL，JALNL，JALS，JALNS，JALO，JALNO
有条件跳转三	JALRE，JALRNE，JALRA，JALRNA，JALRB，JALRNB，JALRG，JALRNG，JALRL，JALRNL，JALRS，JALRNS，JALRO，JALRNO
有条件跳转四	JRE，JRNE，JRA，JRNA，JRB，JRNB，JRG，JRNG，JRL，JRNL，JRS，JRNS，JRO，JRNO

表 1: 指令功能分类与助记符

2.3 数据通路

本文实现的单周期 CPU 的数据通路通过**部件冗余**技术，确保所有操作在一个时钟周期内完成。

具体的数据通路如下：

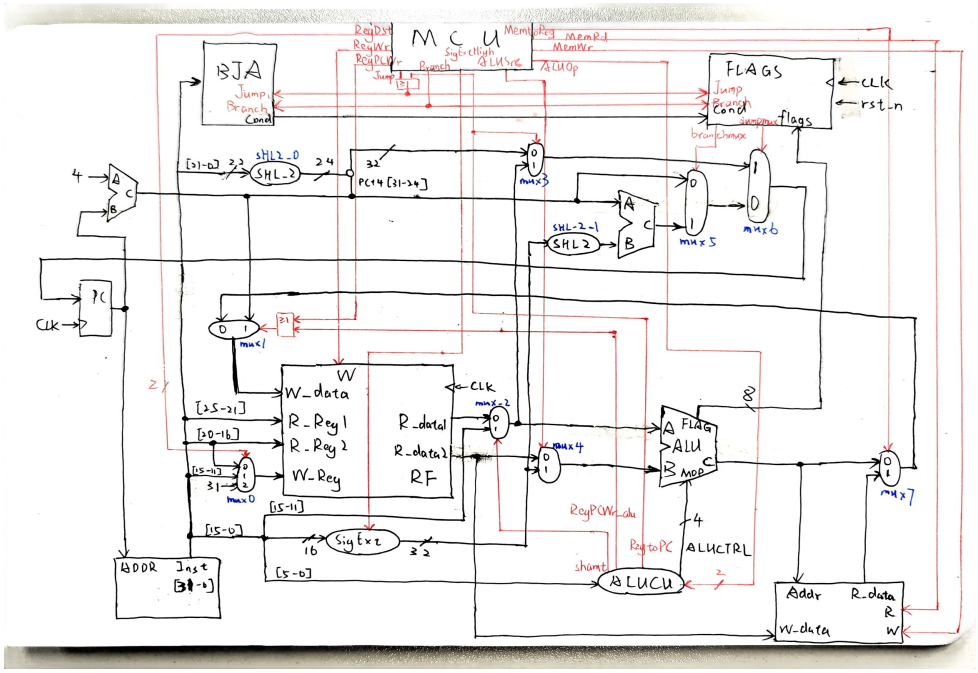

图 1: 单周期 CPU 数据通路

2.4 补充说明

这里详细说明本文在实现单周期 CPU 时，为了对指令集进行扩展，添加的两个**自定义模块**：**BJA** 与 **FLAGS**。

- **BJA** 模块：用于输出**跳转条件码**，根据 *IR* 中的指令机器码，以及 *Jump* 和 *Branch* 两个控制信号，输出对应的跳转条件码。
- **FLAGS** 模块：用于**控制多路选择器，实现多种条件跳转**。*FLAGS* 会接受 *BJA*、*ALU*、控制信号和时钟信号的输入，分析出指令的跳转条件并判断条件是否满足，若满足，则控制多路选择器返回 *PC* 的数据，实现条件跳转。

2.5 测试结果

本文设计了多个测试脚本，用于验证单周期 CPU 的功能。测试脚本包括**基本指令测试、跳转指令测试、有条件跳转指令测试**等。

每个测试脚本都包含**指令序列**和**预期输出**，通过与模拟器的输出进行比较，验证指令是否按预期执行。受限于篇幅限制，仅展示其中一个测试结果，完整的测试内容详见 github 仓库中的结果。

下面是测试脚本：

Listing 1: 单周期 MIPS 测试代码

```
.text
    .globl main

main:
    addi $v1, $zero, 28
    addi $s0, $zero, 0
    addi $s1, $zero, 1
    beq $s0, $s1, target
    jr $v1
    addi $ra, $zero, 64
target:
    addi $s2, $zero, 128
    addi $s1, $zero, 256
```

下面是测试脚本对应的单周期 CPU 的波形输出：

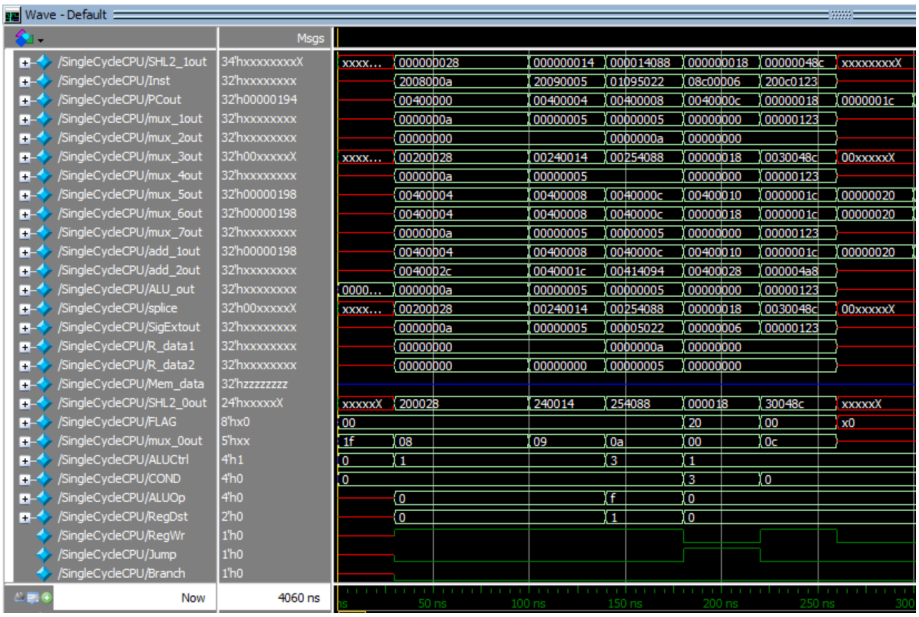

图 2: 单周期 CPU 测试波形

3 多周期 CPU 实现

3.1 多周期 CPU 简介

多周期 CPU 是指每条指令在**多个时钟周期**内完成**取指**、**译码**、**执行**、**访存**、**写回**五个阶段。其核心思想是将指令的执行过程分解为多个阶段，每个阶段占用一个时钟周期，从而提高指令的执行效率。

本文中实现的多周期 CPU 支持 *MIPS* 指令集中的大部分指令，并做了一定的**扩展**。数据通路的实现在单周期 CPU 的数据通路2.4上进行，在多周期的条件下，主要对控制信号和部件有一定的修改，详见3.4。

3.2 实现的指令集

本文实现的多周期 CPU 数据通路同样支持大部分 *MIPS* 指令集中的大部分指令，同时扩展了一部分指令，主要集中在条件转移部分，与单周期 CPU 中支持的指令集**完全相同**，共支持 **89 条指令**，详见2.2。

3.3 数据通路

本文中实现的多周期 CPU 的数据通路实现了使用尽量少的部件，实现尽量多的指令。其中合并、删去、添加的部件和控制信号详见3.4。

具体的数据通路如下图所示：

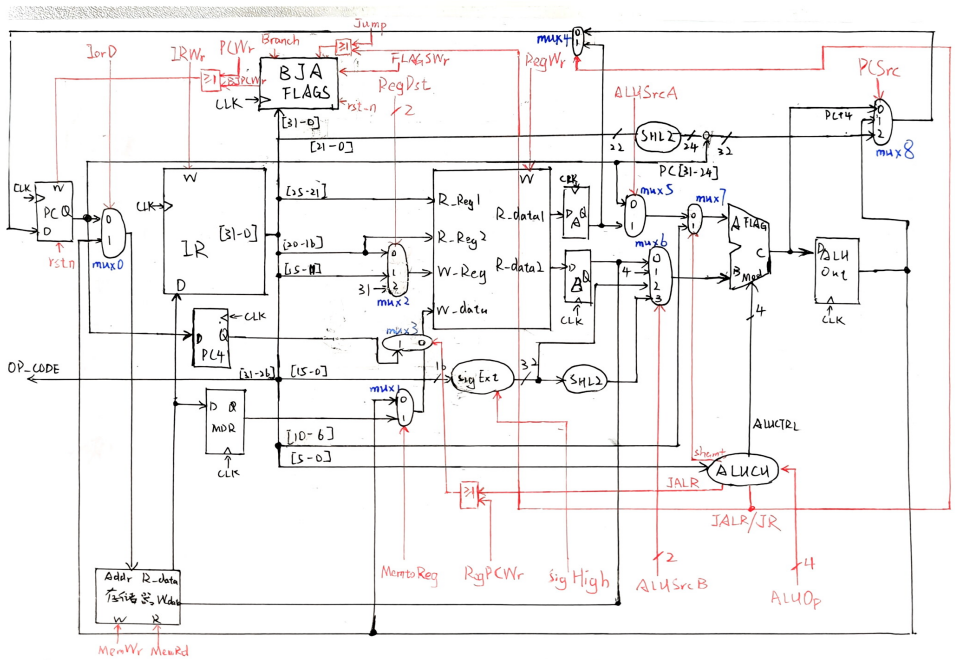

图 3: 多周期 CPU 数据通路

3.4 补充说明

多周期 CPU 数据通路相对于单周期 CPU 数据通路，主要的变动如下：

- 合并数据、指令存储器为主存储器。
- 添加了三路选择器和四路选择器。
- 将原本的控制信号 *ALUSrc* 一分为二，分别为 *ALUSrcA* 和 *ALUSrcB*，用于选择 *ALU* 的输入源。
- 添加了自定义模块 *BJA* 和 *FLAGS*，与单周期 CPU 的数据通路相同，详见2.4。

还有部分微小变动这里并未列出。

3.5 测试结果

同样，本文设计了多种测试脚本对多周期 CPU 进行测试，限于篇幅，仅展示其中一个，完整的测试内容详见 github 仓库中的结果。下面是测试脚本：

Listing 2: 多周期 MIPS 测试代码

```
.text
    .globl main

main:
    addi $s1, $zero, 100
    jala target
    addi $s2, $zero, 64
    jr $ra
target:
    addi $s2, $zero, 256
```

下面是测试脚本对应的流水线 CPU 的波形输出：

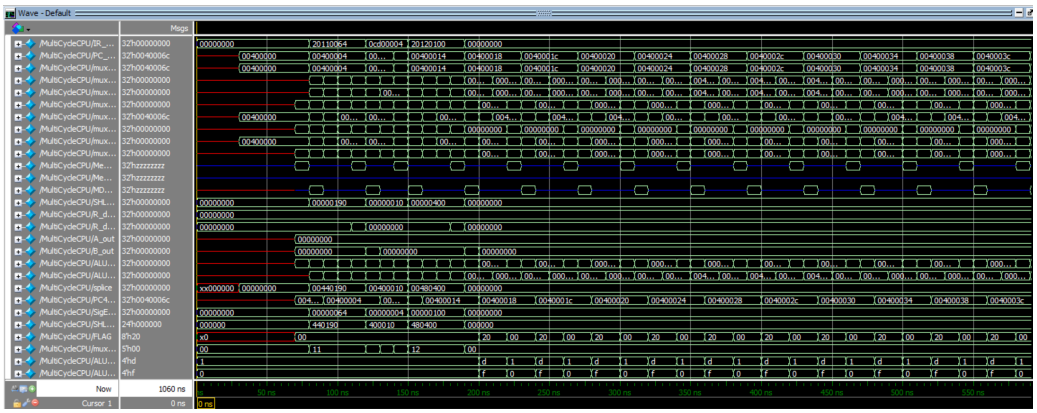

图 4: 多周期 CPU 测试波形

4 流水线 CPU 实现

4.1 流水线 CPU 简介

CPU 的流水线技术是一种通过将指令执行过程分解为多个阶段，从而实现多条指令**并行处理**的技术，显著提高了处理器的性能。

本文实现的流水线 CPU 是将一条指令的执行过程分解为了 5 个阶段，也可以称为是“五级流水 CPU”，5 个阶段分别为：**取指令（FI）、指令译码（ID）、执行指令（EX）、存储器访问（MA）和数据写回（WB）**。

设计流水线 CPU 的核心思想是添加**中间寄存器**，具体的数据通路详见4.3。

4.2 实现的指令集

功能分类	助记符与功能
加载	LW（加载字）
保存	SW（存储字）
R–R 运算	ADD（加），ADDU（无符号加），SUB（减），SUBU（无符号减），SLL（逻辑左移），SRL（逻辑右移），SRA（算术右移），AND（与），OR（或），XOR（异或），NOR（或非），SLT（有符号小于置 1），SLTU（无符号小于置 1）
R–I 运算	ADDI（加立即数），ADDIU（无符号加立即数），ANDI（与立即数），ORI（或立即数），XORI（异或立即数），LUI（加载立即数至高位），SLTI（小于立即数置 1），SLTIU（无符号小于立即数置 1）
分支	BEQ（等于 0 则分支），BNE（不等于 0 则分支），BLEZ（小于等于 0 则分支），BGTZ（大于 0 则分支），BLTZ（小于 0 则分支），BGEZ（大于等于 0 则分支）
条件跳转	J，JE，JNE，JA，JNA，JB，JNB，JG，JNG，JL，JNL，JS，JNS，JO，JNO

表 2: 指令功能分类与助记符

4.3 数据通路

本文中实现的流水线 CPU 数据通路是在2.3中设计的单周期 CPU 数据通路的基础上，添加中间寄存器完成的，同时还添加了新的自定义部件 *DA* 和 *Block3*，分别解决用来解决**数据冒险**和**控制冒险**。对于自定义部件的详细介绍见4.4。

具体的数据通路如下图所示：

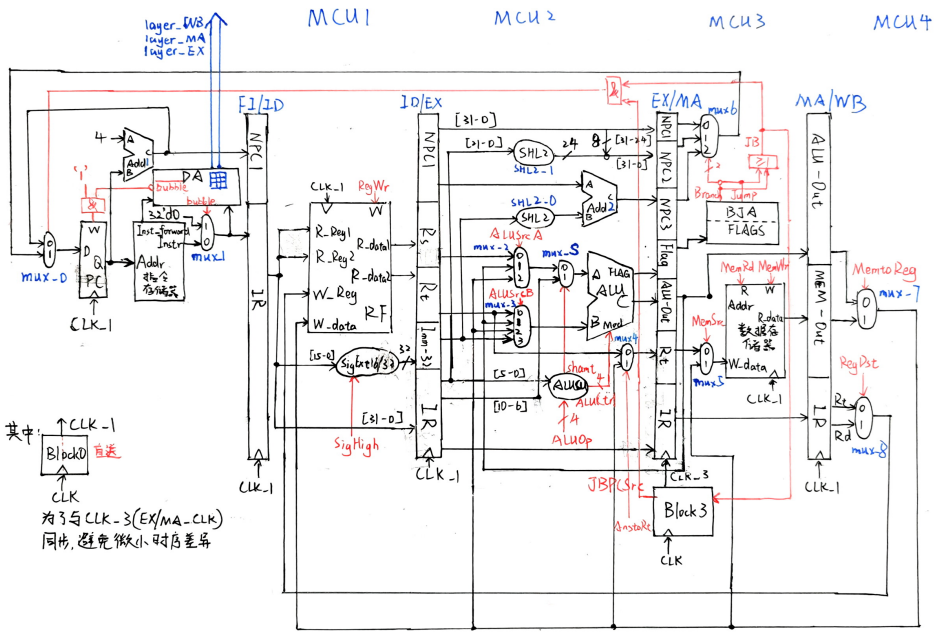

图 5: 流水线 CPU 数据通路

4.4 补充说明

这里详细说明本文在实现流水线 CPU 时，为了解决数据冒险和控制冒险，分别添加的两个**自定义模块**：**DA** 与 **Block3**。

- **DA** 模块：用于解决**数据冒险**。设计思路是，考虑每条指令对于寄存器堆的访问与修改，每条指令最多对寄存器堆的进行两次访问和最多一次修改，因此，*DA* 中记录了处于 *EX*、*MA*、*WB* 状态的三条指令的访问和修改的**寄存器号**，维护一个 3×3 的数组。记录了访问和修改的寄存器后，通过 3×3 的矩阵即可判断是那种类型的数据冒险，并做出对应的控制。

- **Block3** 模块：用于解决**控制冒险**。设计思路是，当遇到**跳转指令**的时候，如果跳转条件成立，则对 *EX* 阶段和 *MA* 阶段的中间寄存器进行**阻塞**，连续阻塞若干周期后，即可令刚刚跳转指令**紧随其后的指令“消耗殆尽”**从而直接执行跳转目标地址的指令。

4.5 冒险情况的解决思路

对于流水线 CPU 而言，可以将所有流水线冒险分为 3 类：**数据冒险**、**控制冒险**和**结构冒险**。下面详细阐述本实验中解决三种冒险的具体方法。

- **数据冒险**：本文解决数据冒险是通过结合“后退法”和设置带旁路技术的 ALU 来进行解决。其中，是否以及选择哪个旁路，以及何时插入气泡，都由 *DA* 模块来完成，注意此时一次性解决了**连续两条**和**连续三条**指令带来的数据冒险，由于本文实现的是五级流水线 CPU，因此不需要考虑**连续四条指令**的数据冒险。

- **控制冒险**：控制冒险是在跳转指令执行时发生的，本文的解决方法是直接的**阻塞**，同时，考虑到如果在跳转指令之后紧跟着的指令是**需要插入气泡解决数据冒险的指令**，而多出的气泡则原解决框架下无法预知，因此，这里采取了**在跳转指令后，阻塞四个时钟周期，相当于令跳转地址的指令在正式执行前空转一次，再次基础上对指令进行“提前取指令”，判断是否有气泡，二者结合便解决了控制冒险的所有情况**，

- **结构冒险**：结构相关的问题通过资源重复或者“部件冗余”的方法进行解决，即这一部分的解决方法就展示在了数据通路当中，这里不再赘述。

4.6 测试结果

同样，本文设计了多种测试脚本对流水线 CPU 进行测试，限于篇幅，仅展示其中一个，完整的测试内容详见 github 仓库中的结果。

下面是测试脚本：

Listing 3: 流水线 MIPS 测试代码

```
.text
    .globl main
main:
    addi $s1, $zero, 16
    addi $s2, $zero, 32
    add $t0, $s1, $s2
    sub $s0, $t0, $s1
    sub $v1, $t0, $s0
```

下面是测试脚本对应的流水线 CPU 的波形输出：

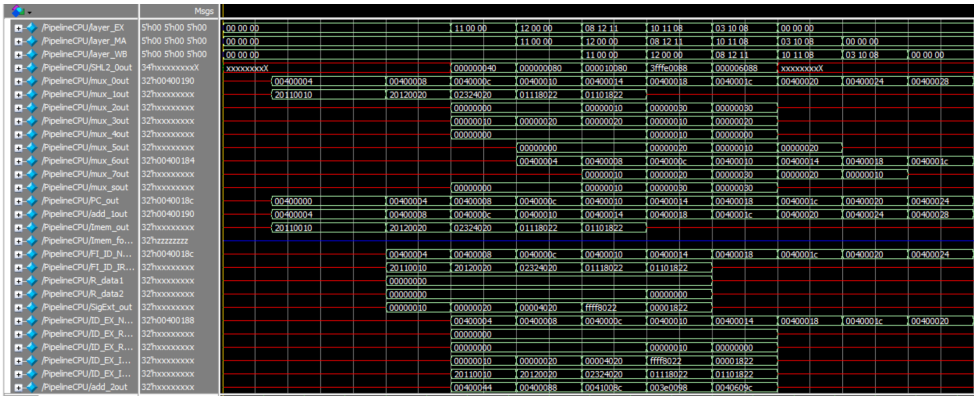

图 6: 流水线 CPU 测试波形